

KOTLIN PROJECT

TRAVEL BOOKING SYSTEM

TEAM MEMBERS

NAGALAKSHMI M
MAHESHWARI P
RANJITH C

KOTLIN INHERITANCE

- » **Inheritance is one of the key features of object-oriented programming.**
- » **It allows user to create a newclass from an existing class.**
- » **The derived class inherits all the features from the baseclass and additional features of its own.**

ADVANTAGES AND DISADVANTAGES

» Easy to Understand:

Simple structure with separate parts (name, seat, etc.).

» Reusability:

TicketBooking class reused across different classes.

» Clear Output:

Labeled and easy-to-read information

» Predefined Values:

The details like name and seat number are fixed in the program. It would be better to allow user input to make the program more flexible.

» No Error Checking:

The program doesn't check for wrong or invalid inputs, which might cause issues in a real-world system.

STEPS INVOLVED

Step 1:

- Create Base Class
- Define a base class TicketBooking.
- Establish basic properties like passenger name, seat number, and travel details.

Step 2: Set Up Inheritance

- Create derived classes that inherit from TicketBooking.
- Add functions to display passenger details (name, seat number, etc.).

Step 3: Object Creation.

Step 4: Implement Methods

- Implement and call methods to print information:
- passengerName.Name()
- seatNo.SeatNo()
- arrivalLocation.ArrivalLocation()
- departureLocation.DepartureLocation()
- arrivalTime.ArrivalTime()
- departureTime.DepartureTime()

Step 5: Final Output

- Display message: "Ticket Booked Successfully"


```
1  @ open class TicketBooking(val name: String) {
2      fun book() {
3          println("Travels: $name")
4      }
5  }
6
7  class PassengerName(name: String) : TicketBooking(name) {
8      fun Name() {
9          println("Passenger Name: $name")
10     }
11 }
12
13 class SeatNo(name: String, val seatNo: Int) : TicketBooking(name) {
14     fun SeatNo() {
15         println("Seat Number: $seatNo")
16     }
17 }
18
19 class ArrivalLocation(name: String, val location: String) : TicketBooking(name) {
20     fun ArrivalLocation() {
21         println("Arrival Location: $location")
22     }
23 }
24
25 class DepartureLocation(name: String, val location: String) : TicketBooking(name) {
26     fun DepartureLocation() {
27         println("Departure Location: $location")
28     }
29 }
30
```

```
31 class ArrivalTime(name: String, val time: String) : TicketBooking(name) {
32     fun ArrivalTime() {
33         println("Arrival Time: $time")
34     }
35 }
36
37 class DepartureTime(name: String, val time: String) : TicketBooking(name) {
38     fun DepartureTime() {
39         println("Departure Time: $time")
40     }
41 }
42
43
44 fun main() {
45     val passengerName = PassengerName(name: "Ranjith")
46     val seatNo = SeatNo(name: "Ranjith", seatNo: 25)
47     val arrivalLocation = ArrivalLocation(name: "Ranjith", location: "Namakkal")
48     val departureLocation = DepartureLocation(name: "Ranjith", location: "Chennai")
49     val arrivalTime = ArrivalTime(name: "Ranjith", time: "10:00 AM")
50     val departureTime = DepartureTime(name: "Ranjith", time: "7:00 AM")
51
52     passengerName.Name()
53     seatNo.SeatNo()
54     arrivalLocation.ArrivalLocation()
55     departureLocation.DepartureLocation()
56     arrivalTime.ArrivalTime()
57     departureTime.DepartureTime()
58     println("TICKET BOOKED")
59 }
```

```
C:\Users\nagal\.jdk\openjdk-22.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.2.1\lib\idea_rt.jar=50402:C:\Program Files\JetB
```

```
Passenger Name: Ranjith
```

```
Seat Number: 25
```

```
Arrival Location: Namakkal
```

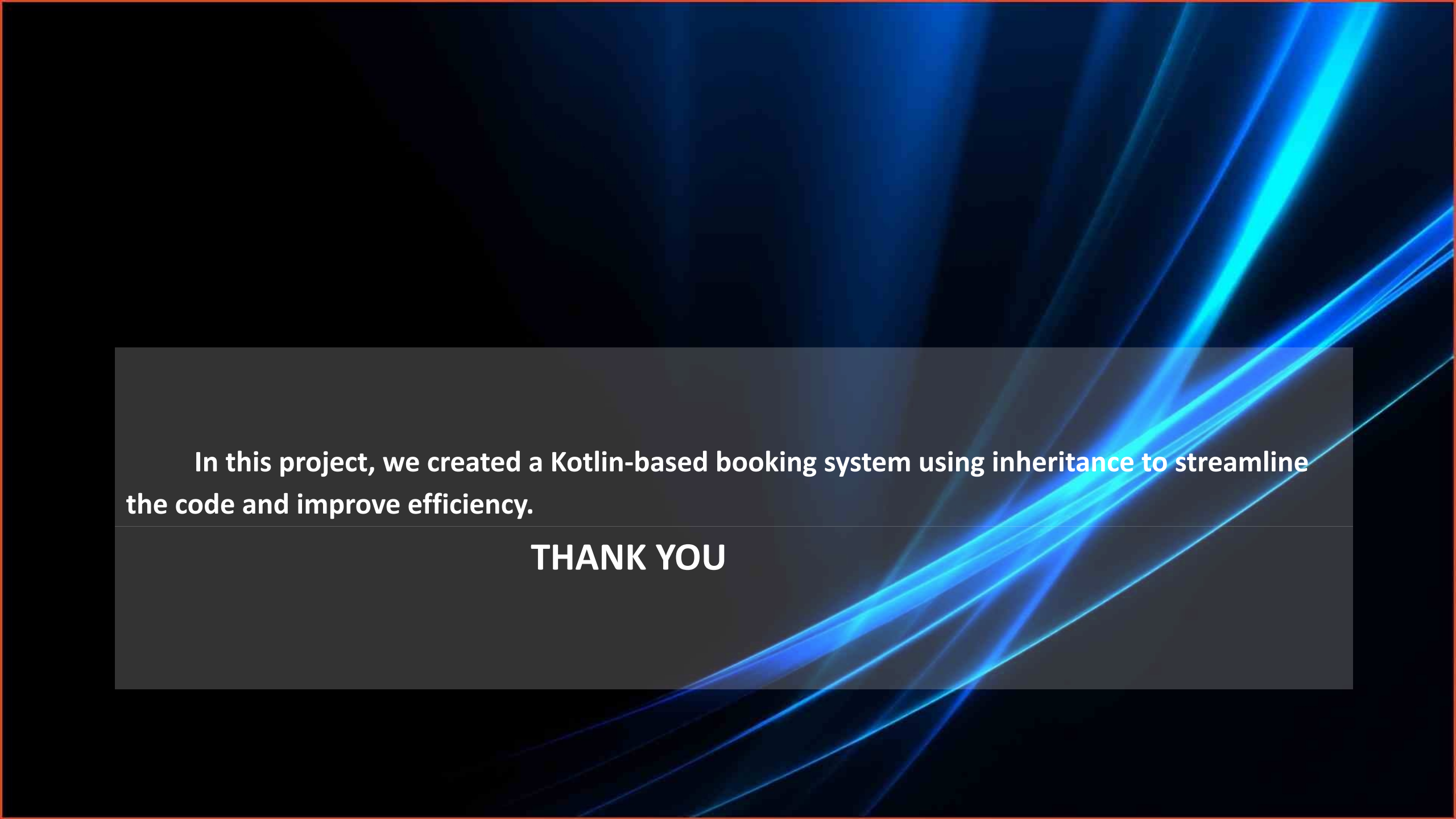
```
Departure Location: Chennai
```

```
Arrival Time: 10:00 AM
```

```
Departure Time: 7:00 AM
```

```
TICKET BOOKED
```

```
Process finished with exit code 0
```



In this project, we created a Kotlin-based booking system using inheritance to streamline the code and improve efficiency.

THANK YOU